存活分析程式 km_analysis.py

Kaplan-Meier／Log-rank／Breslow／Tarone-Ware／Cox（Efron、Breslow）／RMST　執行：python km_analysis.py "Survival test example.xlsx" output

```python
"""
Kaplan-Meier / Log-rank / Cox PH analysis for "Survival test example.xlsx"
Dependencies: numpy, pandas, scipy, matplotlib, seaborn, openpyxl
(lifelines was unavailable, so all estimators are implemented explicitly below.)

Usage: python km_analysis.py "Survival test example.xlsx" outdir
"""
import sys, json, os
import numpy as np
import pandas as pd
from scipy import stats, optimize
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
import seaborn as sns

IN = sys.argv[1] if len(sys.argv) > 1 else "Survival test example.xlsx"
OUT = sys.argv[2] if len(sys.argv) > 2 else "output"
os.makedirs(OUT, exist_ok=True)
Z = stats.norm.ppf(0.975)

# ---------------------------------------------------------------- data
raw = pd.read_excel(IN)
df = raw.rename(columns={raw.columns[0]: "ID", raw.columns[1]: "group",
                         raw.columns[2]: "time", raw.columns[3]: "event"})
df["group"] = df["group"].astype(int)
df["event"] = df["event"].astype(int)
groups = sorted(df["group"].unique())
LABEL = {1: "安慰劑", 2: "新藥"}   # 1 = placebo, 2 = new drug (confirmed by user)


# ---------------------------------------------------------------- Kaplan-Meier
def kaplan_meier(t, e):
    """KM estimate with Greenwood variance and log(-log) 95% CI."""
    t, e = np.asarray(t, float), np.asarray(e, int)
    rows, S, gw = [], 1.0, 0.0
    for u in np.unique(t):
        n = (t >= u).sum()
        d = ((t == u) & (e == 1)).sum()
        c = ((t == u) & (e == 0)).sum()
        if d > 0:
            S *= 1 - d / n
            gw += d / (n * (n - d)) if n > d else np.inf
        if 0 < S < 1 and np.isfinite(gw):
            se_ll = np.sqrt(gw) / abs(np.log(S))          # SE of log(-log S)
            lo, hi = S ** np.exp(Z * se_ll), S ** np.exp(-Z * se_ll)
        elif S == 1:
            lo = hi = 1.0
        else:
            lo = hi = np.nan if S > 0 else 0.0
        se = S * np.sqrt(gw) if np.isfinite(gw) else (0.0 if S == 0 else np.nan)
        rows.append(dict(time=u, n_risk=n, n_event=d, n_censor=c,
                         surv=S, se=se, lower=lo, upper=hi))
    return pd.DataFrame(rows)


def km_quantile(tab, col, p=0.5):
    """Smallest time at which curve `col` is <= p (NaN if never reached)."""
    hit = tab.loc[tab[col] <= p, "time"]
    return float(hit.iloc[0]) if len(hit) else np.nan


km = {g: kaplan_meier(df.loc[df.group == g, "time"], df.loc[df.group == g, "event"])
      for g in groups}

medians = {}
for g in groups:
    tab = km[g].dropna(subset=["surv"])
    # Brookmeyer-Crowley-type CI: times where CI band crosses 0.5
    medians[g] = dict(median=km_quantile(tab, "surv"),
                      lower=km_quantile(tab, "lower"),
                      upper=km_quantile(tab, "upper"))

# ---------------------------------------------------------------- Log-rank
def logrank(df):
    t, e, g = df.time.values, df.event.values, df.group.values
    g1 = groups[0]
    O = E = V = 0.0
    for u in np.unique(t[e == 1]):
        at = t >= u
        n, n1 = at.sum(), (at & (g == g1)).sum()
        d = ((t == u) & (e == 1)).sum()
        d1 = ((t == u) & (e == 1) & (g == g1)).sum()
        O += d1
        E += d * n1 / n
        if n > 1:
            V += d * (n1 / n) * (1 - n1 / n) * (n - d) / (n - 1)
```

```
    chi2 = (O - E) ** 2 / V
    tot = {gg: int(((g == gg) & (e == 1)).sum()) for gg in groups}
    E2 = sum(tot.values()) - E
    return dict(O1=O, E1=E, O2=tot[groups[1]], E2=E2, var=V, chi2=chi2,
                p=stats.chi2.sf(chi2, 1),
                hr_mh=np.exp((tot[groups[1]] - E2) / V),  # Peto/MH approx (grp2 vs grp1)
                )

lr = logrank(df)

def weighted_test(df, wfun):
    t, e, g = df.time.values, df.event.values, df.group.values
    U = V = 0.0
    for u in np.unique(t[e == 1]):
        at = t >= u
        n, n1 = at.sum(), (at & (g == groups[0])).sum()
        d = ((t == u) & (e == 1)).sum()
        d1 = ((t == u) & (e == 1) & (g == groups[0])).sum()
        w = wfun(n)
        U += w * (d1 - d * n1 / n)
        if n > 1:
            V += w * w * d * (n1 / n) * (1 - n1 / n) * (n - d) / (n - 1)
    return dict(chi2=U * U / V, p=stats.chi2.sf(U * U / V, 1))
tests = {"Log-rank (Mantel-Cox)": dict(chi2=lr["chi2"], p=lr["p"]),
         "Breslow (Generalized Wilcoxon)": weighted_test(df, lambda n: n),
         "Tarone-Ware": weighted_test(df, lambda n: np.sqrt(n))}

# ---------------------------------------------------------------- Cox PH (Efron ties)
x = (df.group.values == groups[1]).astype(float)       # 1 = group 2, 0 = group 1 (reference)
tt, ee = df.time.values, df.event.values


def cox_efron(beta, breslow=False):
    ll = U = I = 0.0
    for u in np.unique(tt[ee == 1]):
        risk = tt >= u
        dmask = (tt == u) & (ee == 1)
        d = dmask.sum()
        r = np.exp(beta * x)
        s0, s1, s2 = r[risk].sum(), (r * x)[risk].sum(), (r * x * x)[risk].sum()
        t0, t1, t2 = r[dmask].sum(), (r * x)[dmask].sum(), (r * x * x)[dmask].sum()
        ll += beta * x[dmask].sum()
        U += x[dmask].sum()
        for l in range(d):
            f = 0.0 if breslow else l / d
            a0, a1, a2 = s0 - f * t0, s1 - f * t1, s2 - f * t2
            ll -= np.log(a0)
            U -= a1 / a0
            I += a2 / a0 - (a1 / a0) ** 2
    return ll, U, I


b = 0.0
for _ in range(50):                                    # Newton-Raphson
    _, U, I = cox_efron(b)
    step = U / I
    b += step
    if abs(step) < 1e-10:
        break
ll1, _, I1 = cox_efron(b)
ll0, U0, I0 = cox_efron(0.0)
se_b = 1 / np.sqrt(I1)
cox = dict(beta=b, se=se_b, HR=np.exp(b),
           HR_lo=np.exp(b - Z * se_b), HR_hi=np.exp(b + Z * se_b),
           z=b / se_b, p_wald=2 * stats.norm.sf(abs(b / se_b)),
           LR=2 * (ll1 - ll0), p_LR=stats.chi2.sf(2 * (ll1 - ll0), 1),
           score=U0 ** 2 / I0, p_score=stats.chi2.sf(U0 ** 2 / I0, 1),
           loglik=ll1, loglik0=ll0)
cox["HR_inv"] = 1 / cox["HR"]
cox["HR_inv_lo"], cox["HR_inv_hi"] = 1 / cox["HR_hi"], 1 / cox["HR_lo"]

# profile-likelihood CI (more reliable than Wald with n = 12)
def pl_root(target):
    return lambda bb: 2 * (ll1 - cox_efron(bb)[0]) - stats.chi2.ppf(0.95, 1)
cox["PL_lo"] = np.exp(optimize.brentq(pl_root(None), b - 10, b))
cox["PL_hi"] = np.exp(optimize.brentq(pl_root(None), b, b + 10))

# Breslow ties (SPSS COXREG default) for comparison
bb = 0.0
for _ in range(50):
    _, U, I = cox_efron(bb, True); st = U / I; bb += st
    if abs(st) < 1e-10: break
seb = 1 / np.sqrt(cox_efron(bb, True)[2])
cox["breslow"] = dict(beta=bb, se=seb, HR=np.exp(bb), lo=np.exp(bb - Z * seb), hi=np.exp(bb + Z * seb),
                      p=2 * stats.norm.sf(abs(bb / seb)))

# numerical check of analytic score/information
eps = 1e-5
num_U = (cox_efron(b + eps)[0] - cox_efron(b - eps)[0]) / (2 * eps)
assert abs(num_U) < 1e-5, num_U

# ---------------------------------------------------------------- PH assumption (Schoenfeld / Grambsch-Therneau, Breslow approx)
res, times_ev, Vk = [], [], []
```

```
r = np.exp(b * x)
for i in np.where(ee == 1)[0]:
    risk = tt >= tt[i]
    xbar = (r * x)[risk].sum() / r[risk].sum()
    x2 = (r * x * x)[risk].sum() / r[risk].sum()
    res.append(x[i] - xbar); times_ev.append(tt[i]); Vk.append(x2 - xbar ** 2)
res, times_ev, Vk = map(np.array, (res, times_ev, Vk))
g_ = times_ev - times_ev.mean()
num = (g_ * res).sum() ** 2
den = (g_ ** 2 * Vk).sum() - (g_ * Vk).sum() ** 2 / Vk.sum()
ph = dict(chi2=num / den, p=stats.chi2.sf(num / den, 1))

# ---------------------------------------------------------------- RMST (tau = 7.0)
TAU = float(min(df.groupby("group").time.max()))
def rmst(tab, tau):
    ts = np.r_[0, tab.time.values[tab.time.values < tau], tau]
    ss = np.r_[1, tab.surv.values[tab.time.values < tau]]
    return float((np.diff(ts) * ss).sum())
rm = {g: rmst(km[g], TAU) for g in groups}
rm_spss = {g: rmst(km[g], float(km[g].time.max())) for g in groups}

# ---------------------------------------------------------------- descriptives
desc = df.groupby("group").agg(n=("ID", "size"), events=("event", "sum"),
                               fu_mean=("time", "mean"), fu_median=("time", "median"),
                               fu_min=("time", "min"), fu_max=("time", "max")).reset_index()
desc["censored"] = desc["n"] - desc["events"]

# ---------------------------------------------------------------- figure
fp = [f for f in font_manager.findSystemFonts() if "NotoSansCJK-Regular" in f]
if fp:
    font_manager.fontManager.addfont(fp[0])
    cjk = font_manager.FontProperties(fname=fp[0]).get_name()
else:
    cjk = "sans-serif"
sns.set_theme(style="whitegrid", context="talk",
              rc={"font.family": cjk, "axes.unicode_minus": False})
pal = sns.color_palette("deep")
colors = {groups[0]: pal[0], groups[1]: pal[3]}

fig = plt.figure(figsize=(11, 8.2), dpi=200)
ax = fig.add_axes([0.11, 0.33, 0.85, 0.6])
for g in groups:
    tab = km[g]
    ts = np.r_[0, tab.time.values]
    ss = np.r_[1, tab.surv.values]
    lo = np.r_[1, tab.lower.fillna(0).values]
    hi = np.r_[1, tab.upper.fillna(0).values]
    lo = np.where(np.isnan(lo), 0, lo)
    lab = f"{LABEL[g]}（組別 {g}, n={int((df.group == g).sum())}, 事件={int(df[(df.group == g)].event.sum())}）"
    ax.step(ts, ss, where="post", color=colors[g], lw=2.6, label=lab)
    ax.fill_between(ts, lo, hi, step="post", color=colors[g], alpha=0.13, lw=0)
    cen = tab[tab.n_censor > 0]
    ax.plot(cen.time, cen.surv, "|", color=colors[g], ms=16, mew=2.4)
ax.axhline(0.5, ls="--", lw=1.2, color="0.45")
for g in groups:
    m = medians[g]["median"]
    if np.isfinite(m):
        ax.plot([m, m], [0, 0.5], ls=":", lw=1.6, color=colors[g])
ax.set_xlim(0, 8); ax.set_ylim(-0.02, 1.04)
ax.set_xticks(range(0, 9))
ax.set_xlabel("追蹤時間（月）"); ax.set_ylabel("存活機率 S(t)")
ax.set_title("Kaplan-Meier 存活曲線（陰影：95% CI；| 為設限）", fontsize=17, pad=12)
ax.legend(loc="lower left", fontsize=13, frameon=True)
txt = (f"Log-rank χ² = {lr['chi2']:.2f}, p = {lr['p']:.3f}\n"
       f"HR（新藥 vs 安慰劑）= {cox['HR']:.2f} "
       f"(95% CI {cox['HR_lo']:.2f}–{cox['HR_hi']:.2f})")
ax.text(0.98, 0.97, txt, transform=ax.transAxes, ha="right", va="top", fontsize=12.5,
        bbox=dict(boxstyle="round,pad=0.4", fc="white", ec="0.7"))

# number-at-risk table
axr = fig.add_axes([0.11, 0.06, 0.85, 0.14]); axr.set_xlim(0, 8); axr.set_ylim(-0.5, 1.5)
axr.axis("off")
axr.text(-0.4, 1.95, "風險人數（Number at risk）", fontsize=12.5, ha="left", va="center")
for row, g in enumerate(groups):
    yy = 1 - row
    axr.text(-0.15, yy, f"{LABEL[g]}", ha="right", va="center", fontsize=12.5, color=colors[g],
             fontweight="bold")
    for tick in range(0, 9):
        n = int(((df.group == g) & (df.time >= tick)).sum())
        axr.text(tick, yy, str(n), ha="center", va="center", fontsize=12.5)
fig.savefig(f"{OUT}/km_curve.png", bbox_inches="tight")
plt.close(fig)

# ---------------------------------------------------------------- save results
out = dict(desc=desc.to_dict("records"),
           km={str(g): km[g].to_dict("records") for g in groups},
           medians={str(g): medians[g] for g in groups},
           logrank=lr, tests=tests, rm_spss={str(g): rm_spss[g] for g in groups}, cox=cox, ph=ph, rmst={str(g): rm[g] for g in groups},
tau=TAU,
           data=df.to_dict("records"))
with open(f"{OUT}/results.json", "w") as f:
    json.dump(out, f, ensure_ascii=False, indent=1, default=float)
print(json.dumps({k: out[k] for k in ["medians", "tests", "rm_spss", "cox", "ph", "rmst", "tau"]},
```

```
                indent=1, default=float, ensure_ascii=False))
for g in groups:
    print(g); print(km[g].round(4).to_string(index=False))
```